CSDS 233 Programming Assignment 3
Binary Trees, Expert Systems, and Travel Recommendations

Objectives:
The objectives of this exercise are:

1. To become familiar with the use of the Binary Tree data structure in Java
2. To gain experience saving and reading tree structures to/from files.
3. To gain additional experience in problem-solving.

Assignment Overview:

One achievement in the field of Artificial Intelligence was the development of the area of Expert Systems. One method of implementing an Expert System is to use a Decision Tree. A Decision Tree may be represented as a tree structure. Each node in the tree diagram corresponds to a question and two (or more) possible answers. The user is first asked a question. After choosing a particular response the user is then asked another question, and so on down the branches of the tree to a leaf node, which represents the Expert System's "final answer". In this assignment you will write a program using a very simple Decision Tree to implement an Expert System that can recommend a vacation destination by asking the user questions about her preferences regarding the type of vacation she wishes to take.

Instructions:

The Expert System should make its decision based on the features of the destination which would be significant to the average traveler; these features should include such things as the general climate of the destination (hot, cold), the type of activities available (sun bathing, snow skiing, gambling, etc.), nearby attractions (famous museums, amusement parks, etc.), and so on.

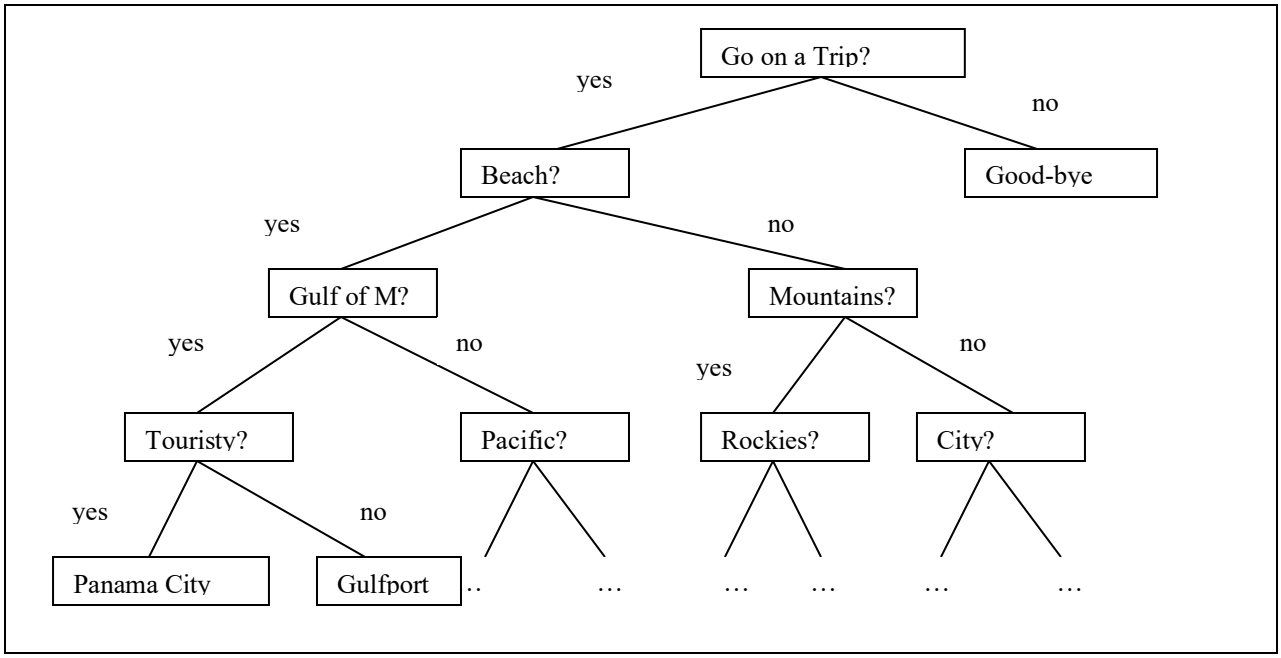
For example, you might ask the user if she prefers to visit some place warm, or some other kind of climate (cold, cool, rainy, sunny, etc.). If she chooses "warm," you might ask if she wants to visit a beach. If she answers "yes," you could ask if she wants to visit the Pacific Ocean. If she answers "yes", you might recommend she visit Hawaii.

Your expert system must include **at least 20 different destinations**. When completed, this might be a nice Expert System to help you, your friends, or your family decide where to go on vacation.

What would the tree look like?

Although decision trees don't have to be binary trees, we will implement this assignment using the BinaryTree nodes we developed in class. One side effect of utilizing binary trees for this problem is that every node can have no more than two branches. Consequently, our expert system can't ask such questions as, "Would you like to go to the mountains, the beach, or a city", because this question would need three branches, and we are limited to a maximum of two. We could ask, "Would you like to go to (a) the beach or the mountains or (b) a city?", and if they picked (a) we could then ask, "Would you like to go to (a) the beach or (b) the mountains?" However, that might strike the user as an artificial way to distinguish between the categories. So, it might be better to make **every question a yes/no question**, such as: "Would you like to go to the beach?"

The binary tree representing our travel expert system might look something like this:



Building/Learning the Decision Tree:

Before we can use the tree, we must build the tree. A binary tree node will have a data field, plus two links: a reference to its left child, and a reference to its right child. You might want to think carefully about what you want to store in the data field of the binary tree node. The data does not have to be numeric; it can be a string, such as a question or the name of a destination.

Every destination node will be a leaf node. A leaf node will have both of its links pointing to null. A parent node will have at least one of its links pointing to a non-null node.

We can build a decision tree by asking an expert yes/no questions. Let's assume that all we have when we start off building a decision tree is just the root node and its two children. The data field of the root node contains the string, "Want to go on a trip?", the "yes" link points to a leaf node containing the string, "Panama City Beach", and the "no" link points to a leaf node containing the string, "Good-bye".

The program can learn about items by executing the following dialog with an expert (the expert's responses are in bold). We begin at the root node and display the "Do you want to go on a trip?" string on the screen. Let's say the item expert answers "y".

```
Let's build an expert system decision tree!
Do you want to go on a trip?
      Y
I'd recommend you going to: Panama City Beach.
Is this where you'd like to go?
```

At this point, if the expert responds "Y", then the Expert System terminates, perhaps with a comment such as, "Thank you for using the Travel Expert System," and then saves the decision tree in a file.

However, if the user responds "N", then this is an opportunity for our expert system to learn something. At this point the expert system should ask:

```
Please tell me where you are thinking of going.
      New York City
Please give me a question that can distinguish between "Panama City
Beach" and "New York City"
      Would you like to go to the beach?
```

```
For "New York City" what would the answer to this question be?
      N
```

Now we can replace the leaf node containing the string, "Panama City Beach" with a parent node. The data field of this node will contain "Would you like go to the beach?", the "Y" link will point to a node whose data field contains "Panama City Beach", and the "N" link will point to a node whose data field contains "New York City".

At this point we have added a new node to the decision tree. We can go back to the root node and repeat the process until we have constructed our full decision tree.

Storing the Decision Tree in a File:

Note that each of the questions in the data field of the decision tree nodes constitutes an attribute that can be used to distinguish among different alternatives. We can represent the item information in the form of an attribute matrix, as follows:

NAME	Attribute 1	Attribute 2	Attribute 3	Attribute 4	Attribute …
Panama City Beach	Y	Y	Y	null	…
New York City	Y	N	N	Y	…
…					…

This suggests that we can store the information that we need to build a decision tree in a text file with a format similar to the following:

"Panama City Beach", Y, Y, Y, null
"New York City", Y, N, Y, N
etc.

Once the data is stored in the file, you can write a function to read in the file and use the information to build a decision tree. You will probably have a better idea about how to store the decision tree; if so, feel free to use it, and clearly specify its format. **The storing and reading in of the tree from a file is an important part of this assignment. While one idea for doing this has been suggested, there are many other, more efficient, possibilities (for example, the pre-order and in-order traversals of a tree stored in a file). It is your responsibility to design an appropriate way to save and read in the tree from file.**

Program specifications:

1) Display a menu allowing the user to select "Build an Expert System", "Find a Destination" or "Quit".
2) If the user chooses "Build an Expert System":
   a) Execute the steps indicated in the Building the Decision Tree section above.
3) If the user chooses "Find a Destination":
   a) If an expert system has not been built, prompt the user for a file name from which to read in the decision tree.
   b) Ask the user the first (or next) question in your decision tree.
   c) Record the user's choice and check to see if the user is trying to enter a choice that violates the rules. If so, request that they reenter the data.
   d) If your Expert System hasn't decided on a destination recommendation, go back to step b). If your Expert System has decided on a recommendation, print out the string stored in its data field, which should be in the following format:
   "I recommend the following Destination:" <name-of-destination>.

d) Go back to step 1).

4) If the user chooses "Quit", display an appropriate message and terminate the program.

You can use any of the binary tree related files that we developed in class as a starting place or you can design your own binary tree structure. However, **this is an individual assignment. You may ask me or the TAs for help, but you may not work with your classmates or a generative AI platform on this problem. In general, if you ever find yourself showing code to another student or looking at the code of another student, then you've gone too far. You may need to look up online or in books the functionality of certain java library methods. If you do this, you must cite this in your code comments and lab report.**

You will also need to provide methods to: make a transition from one node to the next (based on a "yes" or "no" answer), add a question and a result to the tree, write a tree to a file, and read and construct a tree from a file.

Deliverables:

Turn in the following by the due date listed on Canvas:

Upload to Canvas all of the following

- Source code files associated with this assignment.
- Data (i.e., decision tree) files that you generated for this assignment.
- Lab Report which includes these sections:
  - Program Design
  - Answers to the project questions included below
  - Copy of the decision tree file your program generated
  - Test Cases and Test Log

Grading:

This assignment is worth 100 points distributed as follows:

• Build Expert System	25
• Saving/retrieving decision tree to/from file	25
• Expert system correctly making recommendations	20
• Programming/Commenting Style	10
• Report and Project questions	10
• Test cases and test logs	10

Project Questions:

1. What is the algorithmic complexity of your decision tree (justify your answer)? Specify the theoretically largest number of questions that a user might have to answer to receive a guess from the system, given however many items you have in your expert system.
2. Is it possible that you might need to delete a node in a decision tree? Under what circumstances might that occur?
3. Will you need to do a tree traversal for this problem? If so, which type of tree traversal would you recommend?
4. What are the advantages and limitations using a binary tree for this problem? Can you propose a better combination of data structure and algorithm?
5. What was the most interesting part of this assignment?

6. What was the most challenging part of this assignment?
7. How would you like extend your solution (i.e., what other functions would you like to provide for this application)?
8. Any feedback you'd like to provide about the assignment?

Late Policy: Late assignments are penalized at 10% per day:

- Up to 24 hours late--10% off
- 24-48 hours late--20% off
- 48-72 hours late--30% off
- 72-96 hours late--40% off
- 96-120 hours late--50% off
- More than 120 hours late--no longer accepting the assignment